

Laboratório de Banco de Dados

Profa. Msc. Esteic Janaina
Santos Batista

Módulo 2 - Utilização prática de um SGBD usando Funções

Unidade 2 - Triggers



UFMS

AGEAD

Agência de Educação
Digital e a Distância

Introdução às *triggers*

- *Triggers* são pequenas rotinas programadas no banco de dados, semelhantes às conhecidas *Stored Procedures*.
- Porém, como o nome indica (trigger = gatilho), uma *trigger* pode ser disparada em resposta a um determinado evento.
- Geralmente esse evento é uma alteração na tabela, causada pelos comandos *insert*, *delete* ou *update*.

Utilização de *triggers* no PostgreSQL

- Aplicações de *triggers* no PostgreSQL:
 - Manutenção da integridade dos dados.
 - Atualização automática de tabelas.
 - Envio de notificações ou alertas.
 - Realização de auditorias de mudanças no banco de dados.

Criando uma *trigger* no PostgreSQL

- Sintaxe básica de criação de uma *trigger*:

```
CREATE TRIGGER trigger_name  
BEFORE INSERT ON table_name  
FOR EACH ROW  
EXECUTE FUNCTION function_name();
```

Criando uma trigger no PostgreSQL

- Sintaxe básica de criação de uma *trigger*:
 - BEFORE/AFTER: Define se a *trigger* será disparada antes ou depois do evento.
 - FOR EACH ROW: Define que a ação será executada para cada linha afetada.
 - EXECUTE FUNCTION: Chama a função que será executada quando a trigger for disparada.

Tipos de eventos que disparam *triggers*

- Tipos de eventos: INSERT, UPDATE, DELETE
- Exemplos:
 - AFTER INSERT: Uma *trigger* disparada após uma inserção de dados.
 - BEFORE UPDATE: Uma *trigger* disparada antes de uma atualização.

Exemplo prático de *trigger*: auditoria

- Exemplo de *trigger* para auditoria de inserções:

```
CREATE TRIGGER auditar_insercao  
AFTER INSERT ON alunos  
FOR EACH ROW  
EXECUTE FUNCTION log_insercoes();
```

- A função `log_insercoes` grava o evento em uma tabela de auditoria.

Alterar uma *trigger*

- O comando ALTER TRIGGER no PostgreSQL permite alterar o nome de uma *trigger* existente.
- No entanto, ele não permite modificar a lógica interna da *trigger*, como o código da função associada.
- Se você quiser alterar a função que a *trigger* executa, terá que excluir a *trigger* antiga e criar uma nova com a lógica modificada.

Alterar uma trigger

- Suponha que você tenha uma *trigger* chamada `auditar_insercao` na tabela `alunos`, e deseja mudar seu nome para `auditar_novas_insercoes`.
 - `ALTER TRIGGER` `auditar_insercao` ON `alunos` RENAME TO `auditar_novas_insercoes`;

Desativando uma *trigger*

- Se você precisar desativar uma *trigger* temporariamente, o comando **DISABLE TRIGGER** é usado.
- Quando uma *trigger* está desativada, ela não será executada até que seja reativada.

Desativando uma *trigger*

- Vamos desativar a *trigger* auditar_insercao na tabela alunos.

ALTER TABLE alunos **DISABLE TRIGGER** auditar_insercao;

- Você pode reativar a *trigger* usando o comando ENABLE TRIGGER.

ALTER TABLE alunos **ENABLE TRIGGER** auditar_insercao;

Excluindo uma *trigger*

- O comando DROP TRIGGER permite que você remova permanentemente uma *trigger* do banco de dados.
- Vamos remover a trigger auditar_insercao da tabela alunos:

```
DROP TRIGGER IF EXISTS auditar_insercao ON alunos;
```

Excluindo uma *trigger*

- **DROP TRIGGER IF EXISTS:** Remove a *trigger* se ela existir. O uso de IF EXISTS evita um erro caso a *trigger* não exista.
- **auditar_insercao:** Nome da *trigger* a ser excluída.
- **ON alunos:** Especifica a tabela à qual a *trigger* está associada.

FOR EACH STATEMENT

- A cláusula **FOR EACH ROW** não é a única possibilidade ao criar *triggers* no PostgreSQL.
- Além dela, você também pode usar **FOR EACH STATEMENT**.

FOR EACH STATEMENT

FOR EACH ROW:

- **O que faz:** executa a ação da *trigger* para cada linha que é afetada pelo evento (inserção, atualização ou exclusão).
- **Quando usar:** quando você precisa que a *trigger* seja disparada individualmente para cada linha alterada. Por exemplo, ao inserir vários registros de uma vez, a *trigger* será acionada separadamente para cada linha inserida.

FOR EACH STATEMENT

- Neste exemplo, se você inserir 10 registros de uma vez na tabela alunos, a *trigger* será executada 10 vezes, uma para cada linha inserida.

```
CREATE TRIGGER log_insercao_aluno
```

```
AFTER INSERT ON alunos
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION log_insercoes();
```

FOR EACH STATEMENT

FOR EACH STATEMENT:

- **O que faz:** executa a ação da *trigger* apenas uma vez para o comando inteiro, independentemente de quantas linhas foram afetadas.
- **Quando usar:** quando você deseja que a *trigger* seja acionada uma única vez para um comando SQL, mesmo que o comando modifique várias linhas.

FOR EACH STATEMENT

- Neste exemplo, se você inserir 10 registros de uma vez na tabela alunos, a *trigger* será acionada apenas uma vez, independentemente de quantas linhas foram inseridas.

```
CREATE TRIGGER log_insercao_aluno
```

```
AFTER INSERT ON alunos
```

```
FOR EACH STATEMENT
```

```
EXECUTE FUNCTION log_insercoes();
```

Referências



POSTGRESQL. **Tradução da Documentação para o Português do Brasil.** Disponível em: <https://link.ufms.br/8Aaxp>. Acesso em: 05 maio 2024. **Trecho a ser lido: Capítulo 3. Seção 3.2.**

SILBERSCHATZ, Abraham; Korth, Henry F.; Sudarshan, S. **Sistema de Banco de Dados.** Rio de Janeiro: GEN LTC, 2020. ISBN 9788595157552. [Disponível na Biblioteca Digital da UFMS](#). **Trecho a ser lido: Capítulo 5. Seção 5.3.**

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](#).

